

Gruppuppgift

Systemprogrammering och introduktion till C++

Halvtidsrepetition - Kursvecka 1-7

Meddelandeserver med flertrådad arkitektur

Beräknad tid: ca 60 minuter

Gruppstorlek: 4-7 studenter

Introduktion

Denna gruppuppgift är en praktisk repetitionsövning som samlar kunskaper från kursvecka 1 till och med 7. Uppgiften går ut på att tillsammans i gruppen designa och implementera en förenklad meddelandeserver med tillhörande klientprogram. Systemet kombinerar processer, trådar, synkronisering, interprocesskommunikation (IPC) och grundläggande C++-klasser i en realistisk applikation.

Uppgiften är uppdelad i fyra steg som bygger på varandra. Varje steg fokuserar på specifika kunskapsområden och tar ungefär 15 minuter. Gruppen ska fördela arbetet så att alla medlemmar bidrar aktivt. Det är helt i sin ordning att diskutera lösningar inom gruppen - det är faktiskt hela poängen med övningen!

Kunskapsområden som testas

Uppgiften täcker följande områden från kursen:

Vecka	Tema	Tillämpas i steg
1	Processer och systemanrop	Steg 1 - Serverprocessen
2	Trådar och trådprogrammering	Steg 2 - Flertrådad hantering
3	Synkronisering och mutex	Steg 2 - Trådsäker meddelandelista
4	IPC - Pipes	Steg 3 - Loggprocess via pipe
5	IPC - Sockets och delat minne	Steg 1, 3 - Socket-kommunikation
6	Introduktion till C++	Steg 4 - C++-refaktorering
7	Klasser och objektmodellen	Steg 4 - Meddelandeklass

Arbetssätt

Gruppen ska arbeta tillsammans och fördela ansvaret mellan sig. Ett förslag är att dela upp gruppen i par som ansvarar för varsitt steg, men alla ska vara med och diskutera designbeslut. Använd gärna en delad editor (till exempel VS Code Live Share) eller arbeta med Git för att integrera allas bidrag.

Varje steg avslutas med en kort diskussionsfråga som gruppen ska resonera kring tillsammans. Dokumentera era svar kort - detta är bra träning inför den skriftliga reflektionen vid examinationen.

TIPS: Kompilering

C-filer kompileras med:

```
gcc -Wall -pthread -o server server.c
```

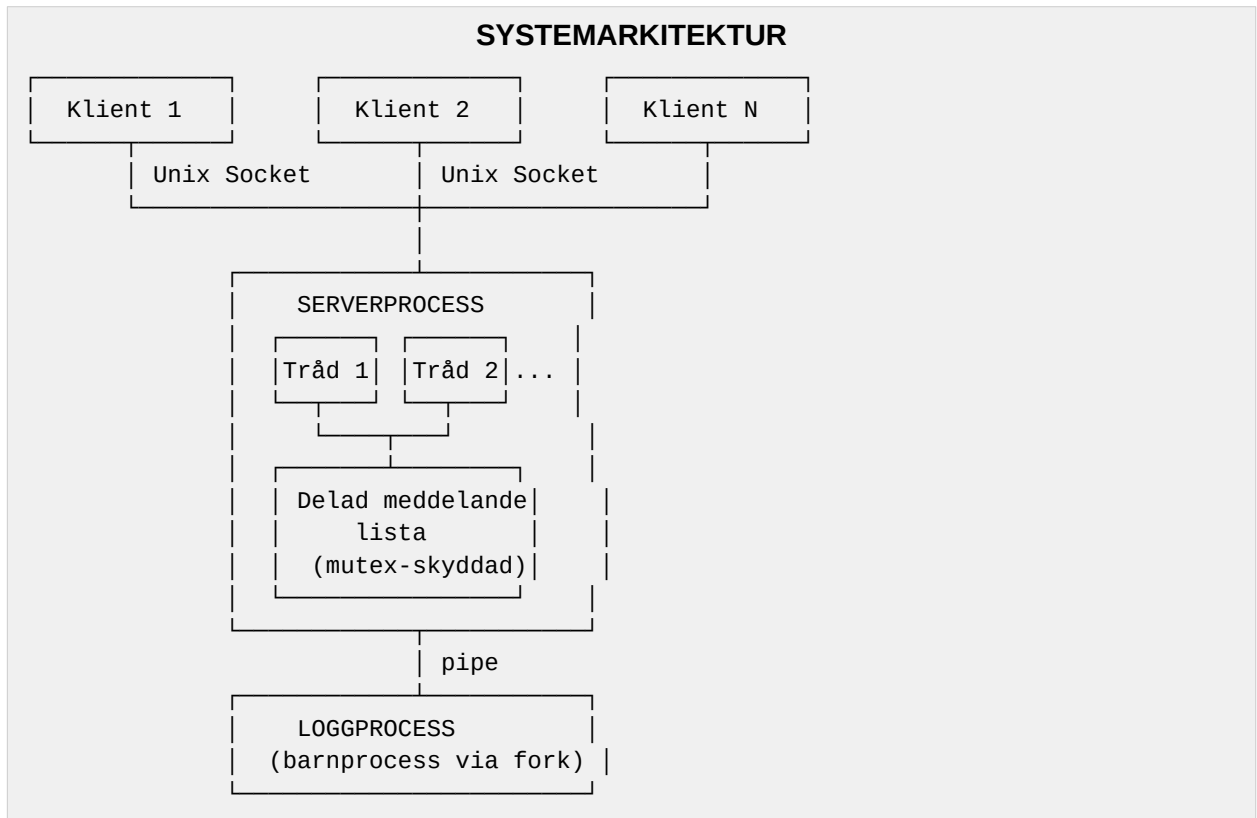
C++-filer kompileras med:

```
g++ -Wall -pthread -std=c++17 -o server server.cpp
```

Systembeskrivning

Systemet vi ska bygga är en förenklad meddelandeserver som fungerar ungefär som en mycket enkel chattapplikation. Servern tar emot meddelanden från klienter via Unix domain sockets, lagrar dem i en delad meddelandelista och skickar vidare meddelanden till alla anslutna klienter. En separat loggprocess tar emot loggmeddelanden via en pipe för att skriva en loggfil.

Arkitekturen ser ut så här i stora drag:



Systemet består av tre huvudkomponenter: en serverprocess som lyssnar på en Unix domain socket och skapar en ny tråd för varje ansluten klient, en delad meddelandelista som skyddas med mutex-lås, och en loggprocess som är en barnprocess skapad med `fork()` som tar emot loggmeddelanden via en pipe. I det sista steget refaktorerar vi delar av koden till C++ med klasser.

Steg 1 - Serverprocess med Unix socket (ca 15 min)

Kunskapsområden: Vecka 1 (processer, fork), Vecka 5 (Unix domain sockets)

I detta första steg skapar vi grundstrukturen för servern. Servern ska skapa en loggprocess med fork() och sedan lyssna på en Unix domain socket för inkommande klientanslutningar. Vi börjar med att hantera en klient i taget (flertrådning kommer i steg 2).

Uppgift 1a: Skapa loggprocessen med fork och pipe

Innan servern börjar lyssna efter klienter ska den skapa en barnprocess som fungerar som en loggskrivare. Kommunikationen mellan serverprocessen (föräldern) och loggprocessen (barnet) sker via en pipe. Loggprocessen ska läsa strängar från pipen och skriva dem till en fil som heter server.log.

Skriv koden i en fil som heter msg_server.c. Implementera följande:

1. Skapa en pipe med pipe() innan fork().
2. Anropa fork() för att skapa loggprocessen.
3. I barnprocessen: stäng skrivänden av pipen, öppna server.log för skrivning, läs i en loop från pipen och skriv varje rad till filen. Avsluta när pipen stängs (read returnerar 0).
4. I föräldraprocessen: stäng läsänden av pipen. Spara skrivandens filbeskrivare i en global variabel (den behövs senare för att skicka loggmeddelanden).

Startkod att utgå ifrån:

```
// msg_server.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <sys/wait.h>
#include <pthread.h>
#include <time.h>

#define SOCKET_PATH "/tmp/msg_server.sock"
#define MAX_MSG_LEN 256
#define MAX_MESSAGES 100
#define MAX_CLIENTS 10

// Global: skrivande av pipe till loggprocessen
int log_pipe_fd = -1;

// Hjälpfunktion: skicka loggmeddelande via pipe
void send_log(const char *msg) {
    if (log_pipe_fd < 0) return;
    char buf[512];
    time_t now = time(NULL);
    struct tm *t = localtime(&now);
    int len = snprintf(buf, sizeof(buf), "[%02d:%02d:%02d] %s\n",
```

```

        t->tm_hour, t->tm_min, t->tm_sec, msg);
    write(log_pipe_fd, buf, len);
}

// TODO: Implementera log_process() som körs i barnprocessen
void log_process(int read_fd) {
    // Steg 1: Öppna server.log för skrivning (append-läge)
    // Steg 2: Läs i en loop från read_fd
    // Steg 3: Skriv varje meddelande till filen
    // Steg 4: Stäng filen och avsluta när pipen stängs
}

int main() {
    int pipe_fds[2];

    // TODO: Skapa pipe
    // TODO: fork() - barnprocessen kör log_process()
    // TODO: I föräldern - stäng läsänden, spara skrivänden

    send_log("Servern startar");

    // Steg 1b fortsätter härifrån...
    return 0;
}

```

Uppgift 1b: Sätt upp Unix domain socket

Nu ska servern börja lyssna på en Unix domain socket. Klienter ansluter till sökvägen /tmp/msg_server.sock. I detta steg accepterar vi bara en klient i taget och skriver ut det meddelande som klienten skickar.

Implementera följande i main() efter pipe/fork-koden:

5. Ta bort eventuell gammal socket-fil med unlink(SOCKET_PATH).
6. Skapa en Unix domain socket med socket(AF_UNIX, SOCK_STREAM, 0).
7. Konfigurera en struct sockaddr_un med sun_family = AF_UNIX och kopiera SOCKET_PATH till sun_path.
8. Anropa bind(), listen() (med MAX_CLIENTS som backlog) och gå in i en accept-loop.
9. I loopen: acceptera en klient, läs ett meddelande med recv(), skriv ut det med printf(), logga det med send_log(), och stäng klientens socket.

Testklient (given kod, spara som msg_client.c):

```

// msg_client.c - Enkel testklient
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>

#define SOCKET_PATH "/tmp/msg_server.sock"
#define MAX_MSG_LEN 256

```

```
int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Användning: %s <meddelande>\n", argv[0]);
        return 1;
    }

    int sock = socket(AF_UNIX, SOCK_STREAM, 0);
    if (sock < 0) { perror("socket"); return 1; }

    struct sockaddr_un addr;
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, SOCKET_PATH, sizeof(addr.sun_path) - 1);

    if (connect(sock, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
        perror("connect");
        close(sock);
        return 1;
    }

    send(sock, argv[1], strlen(argv[1]), 0);
    printf("Skickat: %s\n", argv[1]);

    // Vänta på svar (broadcast-meddelanden)
    char buf[MAX_MSG_LEN];
    int n = recv(sock, buf, sizeof(buf) - 1, 0);
    if (n > 0) {
        buf[n] = '\0';
        printf("Mottaget: %s\n", buf);
    }

    close(sock);
    return 0;
}
```

DISKUSSIONSFRÅGA - Steg 1

Varför använder vi en separat barnprocess för loggningen istället för att bara skriva till filen direkt från serverprocessen? Diskutera fördelar och nackdelar. Tänk på vad som händer om filskrivningen är långsam eller om servern kraschar.

Bonusfråga: Varför är det viktigt att stänga oanvända ändar av pipen i både förälder- och barnprocessen?

Steg 2 - Flertrådad klienthantering (ca 15 min)

Kunskapsområden: Vecka 2 (pthreads), Vecka 3 (mutex, villkorsvariabler)

Nu ska vi utöka servern så att den kan hantera flera klienter samtidigt. Varje klient som ansluter får en egen tråd. Alla trådar delar på en gemensam meddelandelista som måste skyddas med mutex-lås för att undvika race conditions.

Uppgift 2a: Delad meddelandelista med mutex

Implementera en trådsäker meddelandelista som lagrar de senaste meddelandena. Listan är en delad resurs som alla klienttrådar läser från och skriver till, och den måste därför skyddas med ett mutex-lås.

Lägg till följande datastruktur och funktioner i `msg_server.c`:

```
// Delad meddelandelista
typedef struct {
    char messages[MAX_MESSAGES][MAX_MSG_LEN];
    int count;           // Antal meddelanden i listan
    int next_index;      // Nästa position att skriva till (cirkulär)
    pthread_mutex_t lock; // Mutex för trådsäkerhet
} MessageList;

MessageList msg_list = {
    .count = 0,
    .next_index = 0,
    .lock = PTHREAD_MUTEX_INITIALIZER
};

// TODO: Implementera add_message()
// Ska låsa mutex, kopiera meddelandet till rätt plats,
// uppdatera next_index (cirkulärt med %) och count,
// sedan låsa upp mutex.
void add_message(const char *msg) {
    // Din kod här
}

// TODO: Implementera get_latest_message()
// Ska låsa mutex, kopiera det senaste meddelandet till buf,
// sedan låsa upp mutex. Returnera 0 vid lyckad hämtning,
// -1 om listan är tom.
int get_latest_message(char *buf, int buf_size) {
    // Din kod här
    return -1;
}
```

Uppgift 2b: Klienttrådar

Nu ska vi skapa en trådfunktion som hanterar kommunikationen med en enskild klient. Varje gång en ny klient ansluter via `accept()` skapar vi en ny tråd som kör denna funktion. Tråden tar emot meddelanden från klienten, lägger till dem i den

delade meddelandelistan och skickar tillbaka det senaste meddelandet som bekräftelse.

Implementera klienttråden:

```
// Argument till klienttråden
typedef struct {
    int client_fd;    // Klientens socket-filbeskrivare
    int client_id;    // Unikt klient-ID
} ClientArgs;

// TODO: Implementera client_handler()
void *client_handler(void *arg) {
    ClientArgs *args = (ClientArgs *)arg;
    char buf[MAX_MSG_LEN];
    char log_msg[512];

    snprintf(log_msg, sizeof(log_msg),
             "Klient %d ansluten", args->client_id);
    send_log(log_msg);

    // Steg 1: Ta emot meddelande från klienten med recv()
    // Steg 2: Om vi fick data - lägg till i meddelandelistan
    //          med add_message()
    // Steg 3: Logga meddelandet med send_log()
    // Steg 4: Hämta senaste meddelandet och skicka tillbaka
    //          till klienten med send()
    // Steg 5: Stäng klientens socket och frigör minne

    close(args->client_fd);
    free(args);
    return NULL;
}
```

Uppdatera accept-loopen i main() så att den skapar en ny tråd:

```
// Uppdaterad accept-loop i main()
int client_count = 0;
while (1) {
    int client_fd = accept(server_fd, NULL, NULL);
    if (client_fd < 0) {
        perror("accept");
        continue;
    }

    // Allokerar argument på heapen (tråden frigör)
    ClientArgs *args = malloc(sizeof(ClientArgs));
    args->client_fd = client_fd;
    args->client_id = ++client_count;

    pthread_t tid;
    if (pthread_create(&tid, NULL, client_handler, args) != 0) {
        perror("pthread_create");
        close(client_fd);
        free(args);
    }
}
```



```
        continue;
    }
    pthread_detach(tid); // Vi behöver inte joina
}
```

VIKTIGT: Trådsäkerhet

Notera varför vi allokerar ClientArgs med malloc() istället för att använda en lokal variabel: om vi använde en lokal variabel i loopen skulle nästa iteration kunna skriva över den innan tråden hinner läsa värdena. Detta är ett klassiskt race condition-problem! Tråden ansvarar för att frigöra minnet med free().

DISKUSSIONSFRÅGA - Steg 2

Vad skulle hända om vi tog bort mutex-låset i add_message() och get_latest_message()? Beskriv ett konkret scenario där två trådar kan orsaka ett fel (race condition). Tänk steg för steg: vilka instruktioner kör varje tråd och i vilken ordning?

Bonusfråga: Vi använder pthread_detach() istället för pthread_join(). Varför är det lämpligt i detta fall? Vad är skillnaden?

Steg 3 - Utökad loggning med strukturerade meddelanden (ca 15 min)

Kunskapsområden: Vecka 4 (pipes, dup2), Vecka 5 (IPC-jämförelse)

I detta steg förbättrar vi loggprocessen genom att skicka strukturerade loggmeddelanden via pipen. Istället för att bara skicka textsträngar definierar vi ett enkelt binärt protokoll med en struct som innehåller loggnivå, tidsstämpel och meddelande. Vi utökar också loggprocessen med statistik.

Uppgift 3a: Strukturerat loggprotokoll

Definiera en struct för loggmeddelanden och skriv om `send_log()` samt loggprocessen så att de använder denna struct. På så sätt kan vi skicka binär data genom pipen istället för formaterade strängar, vilket ger oss mer flexibilitet.

Definiera följande struct och uppdatera loggfunktionerna:

```
// Loggnivåer
typedef enum {
    LOG_INFO,
    LOG_WARNING,
    LOG_ERROR
} LogLevel;

// Strukturerat loggmeddelande som skickas via pipen
typedef struct {
    LogLevel level;
    time_t timestamp;
    char message[MAX_MSG_LEN];
} LogEntry;

// Namn för loggnivåer (för utskrift)
const char *level_names[] = { "INFO", "WARNING", "ERROR" };

// TODO: Skriv om send_log() så den skickar en LogEntry
void send_log(LogLevel level, const char *msg) {
    if (log_pipe_fd < 0) return;
    LogEntry entry;
    entry.level = level;
    entry.timestamp = time(NULL);
    strncpy(entry.message, msg, MAX_MSG_LEN - 1);
    entry.message[MAX_MSG_LEN - 1] = '\0';

    // TODO: Skriv hela structen till pipen med write()
    // Tänk på: vad ska storleken vara? Använd sizeof(LogEntry)
}
```

Uppgift 3b: Utökad loggprocess med statistik

Skriv om loggprocessen så att den läser `LogEntry`-structer från pipen istället för textsträngar. Loggprocessen ska dessutom hålla räkningen på hur många

meddelanden som loggats per nivå och skriva ut en sammanfattning när pipen stängs (det vill säga när servern avslutas).

Uppdatera log_process():

```
void log_process(int read_fd) {
    FILE *log_file = fopen("server.log", "a");
    if (!log_file) {
        perror("fopen server.log");
        exit(1);
    }

    // Statistikräknare
    int stats[3] = {0, 0, 0}; // INFO, WARNING, ERROR
    LogEntry entry;

    // TODO: Läs LogEntry-structer i en loop
    // Tips: read(read_fd, &entry, sizeof(LogEntry))
    // Kontrollera att read() returnerar sizeof(LogEntry)
    while (read(read_fd, &entry, sizeof(LogEntry)) == sizeof(LogEntry)) {
        // TODO: Formatera tidsstämpel med localtime() och strftime()
        // TODO: Skriv till loggfilen med format:
        //      [HH:MM:SS] [NIVÅ] meddelande
        // TODO: Uppdatera statistikräknaren
        // TODO: Glöm inte fflush() så att data skrivs direkt
    }

    // Skriv sammanfattning
    fprintf(log_file, "\n--- Loggsammanfattning ---\n");
    fprintf(log_file, "INFO: %d, WARNING: %d, ERROR: %d\n",
            stats[LOG_INFO], stats[LOG_WARNING], stats[LOG_ERROR]);
    fprintf(log_file, "Totalt: %d meddelanden\n",
            stats[0] + stats[1] + stats[2]);

    fclose(log_file);
    exit(0);
}
```

Uppdatera alla anrop till send_log() i resten av koden så att de inkluderar en loggnivå:

```
// Exempel på uppdaterade anrop:
send_log(LOG_INFO, "Servern startar");
send_log(LOG_INFO, "Klient ansluten");
send_log(LOG_WARNING, "Maxantal klienter nått");
send_log(LOG_ERROR, "Kunde inte skapa tråd");
```

TIPS: Binär data via pipes

När vi skickar en struct via en pipe skickas rå bytes. Det betyder att vi kan skicka vilken datatyp som helst, inte bara textsträngar. Fördelen är att vi slipper parsning - mottagaren vet exakt var varje fält finns i minnet. Nackdelen är att det bara fungerar mellan processer på samma maskin med samma arkitektur (byte order och padding kan skilja sig mellan plattformar).

En pipe garanterar atomära skrivningar upp till PIPE_BUF (minst 512 bytes, vanligen 4096

bytes på Linux). Vår LogEntry-struct är betydligt mindre än detta, så varje write() kommer att vara atomär.

DISKUSSIONSFRÅGA - Steg 3

Vi skickar nu strukturerad binär data (structer) via pipen istället för textsträngar. Jämför detta med de andra IPC-metoderna ni lärt er: Unix sockets och delat minne. Om vi istället ville att loggprocessen körde på en annan dator, vilken IPC-metod skulle vi behöva använda och varför? Vilken IPC-metod hade varit mest effektiv om båda processerna körde på samma maskin och behövde dela stora datamängder?

Steg 4 - Refaktorering till C++ med klasser (ca 15 min)

Kunskapsområden: Vecka 6 (C++-grunderna, references, namespaces, iostream), Vecka 7 (klasser, konstruktörer, destruktörer, åtkomstkontroll)

I det sista steget tar vi meddelandelistan och kapslar in den i en C++-klass. Vi skriver inte om hela servern till C++ utan fokuserar på att visa hur en C-struct med tillhörande funktioner kan omvandlas till en klass med korrekt inkapsling, konstruktör och destruktör. Detta är en viktig brygga mellan C och C++ som visar fördelarna med objektmodellen.

Uppgift 4a: Klassen MessageStore

Skapa en ny fil som heter `message_store.hpp`. I denna fil ska ni definiera en klass som heter `MessageStore` som ersätter vår C-struct `MessageList` och dess tillhörande funktioner. Klassen ska kapsla in all data och logik för meddelandehantering.

Skapa filen `message_store.hpp` med följande struktur:

```
// message_store.hpp
#ifndef MESSAGE_STORE_HPP
#define MESSAGE_STORE_HPP

#include <cstring>
#include <iostream>
#include <pthread.h>

class MessageStore {
private:
    static const int MAX_MESSAGES = 100;
    static const int MAX_MSG_LEN = 256;

    char messages[MAX_MESSAGES][MAX_MSG_LEN];
    int count;
    int next_index;
    pthread_mutex_t lock;

public:
    // TODO: Implementera konstruktorn
    // Ska initialisera count och next_index till 0
    // Ska initialisera mutex med pthread_mutex_init()
    MessageStore() {
        // Din kod här
    }

    // TODO: Implementera destruktorn
    // Ska förstöra mutex med pthread_mutex_destroy()
    ~MessageStore() {
        // Din kod här
    }

    // Förhindra kopiering (mutex kan inte kopieras)
    MessageStore(const MessageStore&) = delete;
```

```

MessageStore& operator=(const MessageStore&) = delete;

// TODO: Implementera addMessage()
// Samma logik som C-versionen av add_message()
// men nu som en medlemsfunktion
void addMessage(const char *msg) {
    pthread_mutex_lock(&lock);
    // Din kod här - kopiera msg till rätt plats
    // Uppdatera next_index och count
    pthread_mutex_unlock(&lock);
}

// TODO: Implementera getLatestMessage()
// Returnera true om det finns ett meddelande, annars false
bool getLatestMessage(char *buf, int buf_size) {
    pthread_mutex_lock(&lock);
    if (count == 0) {
        pthread_mutex_unlock(&lock);
        return false;
    }
    // Din kod här - kopiera senaste meddelandet till buf
    pthread_mutex_unlock(&lock);
    return true;
}

// TODO: Implementera getMessageCount()
// Trådsäker hämtning av antal meddelanden
int getMessageCount() const {
    // OBS: const-metod men vi behöver låsa mutex.
    // Vi kan använda const_cast här eller göra lock mutable.
    // Diskutera i gruppen: varför är detta problematiskt?
    return count; // Förenklad version utan lås
}

// Skriv ut alla meddelanden (för debugging)
void printAll() const {
    std::cout << "=== Meddelanden (" << count
                << " st) ===" << std::endl;
    int start = (count < MAX_MESSAGES) ? 0
                : next_index;
    int total = (count < MAX_MESSAGES) ? count
                : MAX_MESSAGES;
    for (int i = 0; i < total; i++) {
        int idx = (start + i) % MAX_MESSAGES;
        std::cout << " [" << i + 1 << "] "
                  << messages[idx] << std::endl;
    }
}

};

#endif // MESSAGE_STORE_HPP

```

Uppgift 4b: Testprogram för MessageStore

Skriv ett enkelt testprogram som skapar en MessageStore-instans, lägger till meddelanden från flera trådar och verifierar att allt fungerar korrekt. Detta visar hur klassen kan användas istället för de fristående C-funktionerna.

Skapa filen test_store.cpp:

```
// test_store.cpp
#include "message_store.hpp"
#include <pthread.h>
#include <cstdio>
#include <cstring>

// Delad MessageStore-instans (skapas i main, skickas till trådar)
struct ThreadArgs {
    MessageStore *store; // Referens via pekare
    int thread_id;
};

void *writer_thread(void *arg) {
    ThreadArgs *args = static_cast<ThreadArgs*>(arg);
    char msg[256];

    for (int i = 0; i < 5; i++) {
        snprintf(msg, sizeof(msg), "Tråd %d: meddelande %d",
            args->thread_id, i + 1);
        args->store->addMessage(msg);
        std::cout << "Skickat: " << msg << std::endl;
    }

    // TODO: Hämta och skriv ut senaste meddelandet
    char latest[256];
    if (args->store->getLatestMessage(latest, sizeof(latest))) {
        std::cout << "Tråd " << args->thread_id
            << " senaste: " << latest << std::endl;
    }

    return nullptr; // C++ nullptr istället för NULL
}

int main() {
    // MessageStore skapas på stacken - konstruktorn körs
    MessageStore store;

    const int NUM_THREADS = 4;
    pthread_t threads[NUM_THREADS];
    ThreadArgs args[NUM_THREADS];

    // Skapa trådar
    for (int i = 0; i < NUM_THREADS; i++) {
        args[i].store = &store; // Alla delar samma instans
        args[i].thread_id = i + 1;
        pthread_create(&threads[i], nullptr,
            writer_thread, &args[i]);
    }
}
```

```

}

// Vänta på alla trådar
for (int i = 0; i < NUM_THREADS; i++) {
    pthread_join(threads[i], nullptr);
}

// Skriv ut alla meddelanden
store.printAll();
std::cout << "Totalt antal: " << store.getMessageCount()
           << std::endl;

// Destruktorn körs automatiskt när store går ur scope!
return 0;
}

```

TIPS: Kompilera och testa

Kompilera testprogrammet med:

```
g++ -Wall -pthread -std=c++17 -o test_store test_store.cpp
```

Kör programmet och kontrollera att alla 20 meddelanden (4 trådar × 5 meddelanden) lagras korrekt och att det inte finns några konstiga utskrifter som tyder på race conditions.

DISKUSSIONSFRÅGA - Steg 4

Jämför C-versionen (MessageList-structen med fristående funktioner) med C++-versionen (MessageStore-klassen). Vilka fördelar ser ni med klassversionen? Tänk på åtkomstkontroll (public/private), att konstruktorn och destruktorn körs automatiskt, och att vi förhindrat kopiering med delete. Kan ni komma på situationer där C-versionen ändå kan vara att föredra?

Bonusfråga: Varför har vi tagit bort kopieringskonstruktorn och tilldelningsoperatoren med = delete? Vad skulle hända om någon försökte kopiera en MessageStore?

Lösningsnycklar och ledtrådar

Om gruppen kör fast finns här ledtrådar för varje steg. Försök lösa uppgiften själva först! Ledtrådarna är ordnade från vaga tips till mer specifika lösningar.

Ledtrådar för Steg 1

1a - Loggprocess med pipe:

- Skapa pipen INNAN fork() - båda processerna behöver se filbeskrivarna.
- pipe_fds[0] är läsänden, pipe_fds[1] är skrivänden.
- I barnprocessen: close(pipe_fds[1]) och läs från pipe_fds[0].
- Loopen i barnprocessen: while ((n = read(read_fd, buf, sizeof(buf) - 1)) > 0) { buf[n] = '\0'; fprintf(log_file, "%s", buf); fflush(log_file); }

1b - Unix domain socket:

```
// Kompletta socket-setup i main():
unlink(SOCKET_PATH);

int server_fd = socket(AF_UNIX, SOCK_STREAM, 0);
if (server_fd < 0) { perror("socket"); exit(1); }

struct sockaddr_un addr;
memset(&addr, 0, sizeof(addr));
addr.sun_family = AF_UNIX;
strncpy(addr.sun_path, SOCKET_PATH,
        sizeof(addr.sun_path) - 1);

if (bind(server_fd, (struct sockaddr*)&addr,
        sizeof(addr)) < 0) {
    perror("bind"); exit(1);
}
if (listen(server_fd, MAX_CLIENTS) < 0) {
    perror("listen"); exit(1);
}

send_log(LOG_INFO, "Server lyssnar på socket");
```

Ledtrådar för Steg 2

2a - Trådsäker add_message():

```
void add_message(const char *msg) {
    pthread_mutex_lock(&msg_list.lock);
    strncpy(msg_list.messages[msg_list.next_index],
            msg, MAX_MSG_LEN - 1);
    msg_list.messages[msg_list.next_index][MAX_MSG_LEN - 1]
        = '\0';
    msg_list.next_index =
        (msg_list.next_index + 1) % MAX_MESSAGES;
    if (msg_list.count < MAX_MESSAGES) {
        msg_list.count++;
    }
}
```

```

    }
    pthread_mutex_unlock(&msg_list.lock);
}

```

2b - Klienttrådsfunktion (kärnan):

```

void *client_handler(void *arg) {
    ClientArgs *args = (ClientArgs *)arg;
    char buf[MAX_MSG_LEN];
    char log_msg[512];

    snprintf(log_msg, sizeof(log_msg),
             "Klient %d ansluten", args->client_id);
    send_log(LOG_INFO, log_msg);

    int n = recv(args->client_fd, buf,
                 sizeof(buf) - 1, 0);
    if (n > 0) {
        buf[n] = '\0';
        add_message(buf);

        snprintf(log_msg, sizeof(log_msg),
                 "Klient %d: %s",
                 args->client_id, buf);
        send_log(LOG_INFO, log_msg);

        char latest[MAX_MSG_LEN];
        if (get_latest_message(latest,
                                sizeof(latest)) == 0) {
            send(args->client_fd, latest,
                 strlen(latest), 0);
        }
    }

    close(args->client_fd);
    free(args);
    return NULL;
}

```

Ledtrådar för Steg 3

Skicka struct via pipe:

```

// I send_log() - skriv hela structen:
write(log_pipe_fd, &entry, sizeof(LogEntry));

// I loggprocessen - formatera och skriv:
struct tm *t = localtime(&entry.timestamp);
fprintf(log_file, "[%02d:%02d:%02d] [%s] %s\n",
        t->tm_hour, t->tm_min, t->tm_sec,
        level_names[entry.level], entry.message);
stats[entry.level]++;
fflush(log_file);

```

Ledtrådar för Steg 4

Konstruktör och destruktör:

```
// Konstruktör
MessageStore() : count(0), next_index(0) {
    pthread_mutex_init(&lock, nullptr);
    // Nollställ meddelandearrayen
    memset(messages, 0, sizeof(messages));
}

// Destruktor
~MessageStore() {
    pthread_mutex_destroy(&lock);
    std::cout << "MessageStore förstörd, hade "
                << count << " meddelanden"
                << std::endl;
}
```

getLatestMessage() - hämta senaste:

```
bool getLatestMessage(char *buf, int buf_size) {
    pthread_mutex_lock(&lock);
    if (count == 0) {
        pthread_mutex_unlock(&lock);
        return false;
    }
    int latest = (next_index - 1 + MAX_MESSAGES)
                 % MAX_MESSAGES;
    strncpy(buf, messages[latest], buf_size - 1);
    buf[buf_size - 1] = '\0';
    pthread_mutex_unlock(&lock);
    return true;
}
```

Sammanfattning och checklista

Grattis om ni har kommit igenom alla fyra stegen! Nedan finns en checklista för att kontrollera att ni har täckt alla viktiga delar samt en sammanfattning av vad vi har repeterat.

Checklista

✓	Moment	Koncept
☐	Loggprocess skapad med fork()	fork, processer
☐	Pipe fungerar mellan förälder och barn	pipe, IPC
☐	Unix domain socket skapad och bunden	sockets, bind, listen
☐	Klienter kan ansluta och skicka meddelanden	accept, recv, send
☐	En tråd skapas per klient	pthread_create
☐	Meddelandelistan skyddas med mutex	pthread_mutex
☐	Argument allokeras på heap för trådar	malloc, trådsäkerhet
☐	Strukturerade loggmeddelanden via pipe	struct, binär IPC
☐	Loggstatistik skrivs vid avslut	pipes, formatering
☐	MessageStore-klass med inkapsling	class, private/public
☐	Konstruktor och destruktor fungerar	RAII-tänk
☐	Kopiering förhindrad med = delete	kopieringskontroll
☐	Testprogram med flertrådad åtkomst	C++ + pthreads
☐	Diskussionsfrågor besvarade (4 st)	reflektion

Koppling till kursmål

Denna gruppuppgift har berört följande kursmål direkt:

Kursmål	Beskrivning
1	Förklara hur operativsystem hanterar processer, trådar, synkronisering och minne.
2	Redogöra för interprocesskommunikation som pipes, sockets och delat minne.
3	Förklara skillnader mellan C och C++ i syntax, abstraktionsnivå och resursmodell.
4	Redogöra för C++-objektmodellen och RAII som metod för resursförvaltning.
7	Implementera flertrådade C- eller C++-program med effektiv synkronisering och korrekt resursdelning.

8	Använda IPC-lösningar för processkommunikation i systemnära program.
---	--

Tips för vidare lärande

De kommande veckorna i kursen bygger vidare på det vi har repeterat här. I vecka 8 kommer vi att lära oss om RAII på riktigt, och vår MessageStore-klass är ett bra exempel på varför automatisk resurshantering (destruktor som städar upp mutex) är så värdefullt. I vecka 9 kommer vi att ersätta vår handskrivna cirkulära buffert med STL-containers som `std::vector` och `std::string`, vilket kommer att förenkla koden avsevärt.

Tänk på att diskussionsfrågorna i denna övning är utmärkta förberedelser inför den skriftliga reflektionen som är en del av examinationen. Spara gärna era anteckningar!

Bra jobbat!

Ni har nu repeterat och tillämpat koncept från sju veckors systemprogrammering i ett sammanhängande system. Det här är precis den typen av integration av kunskap som krävs för att bygga riktiga systemnära applikationer. Fortsätt öva och experimentera med koden - prova att utöka systemet med fler funktioner, till exempel broadcast till alla anslutna klienter eller ett kommandogränssnitt.